ĐÁP ÁN MÔN: CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Nội dung 1:

Câu 1:

1a:

<u>Bước 1.1:</u>

Chọn tùy ý một phần tử a[k] trong dãy, L≤k≤R

x=a[k], i=L, j=R

<u>Bước 1.2:</u>

Phát hiện và hiệu chỉnh cặp a[i] và a[j] nằm sai chỗ:

Bước 1.2a: Trong khi (a[i]<x) i++

Bước 1.2b: Trong khi (a[j]>x) j--

Bước 1.2c: Nếu (i≤j): Hoán vị a[i] và a[j]; i++, j--

<u>Bước 1.3:</u>

Nếu i<j: Lặp lại bước 1.2

Ngược lại: Dừng phân hoạch

1b:

Bước 1:

Key 38	**38**	**81**	**22**	**48**	**13**	**69**	**93**	**14**	**45**

Bước 2:

Key 22	**22**	**13**	**14**	**38**	**Key 81**	**81**	**48**	**69**	**95**	**45**

Bước 3:

Key 13	**13**	**14**	**22**	**38**	**Key 48**	**48**	**69**	**45**	**81**	**95**

Bước 4:

Key 14	**13**	**14**	**22**	**38**	**Key 45**	**45**	**48**	**69**	**81**	**95**

Bước 5:

	Key 22	**13**	**14**	**22**	**38**	**Key 69**	**45**	**48**	**69**	**81**	**95**

Bước 6:

Key 22	**13**	**14**	**22**	**38**	**Key 95**	**45**	**48**	**69**	**81**	**95**

Dãy đã sắp xếp tăng dần:

13	**14**	**22**	**38**	**45**	**48**	**69**	**81**	**95**

Câu 2:

2a:

```
void LNR(TREE T)
{
    if(T != NULL)
    {
        LNR(T -> pLeft);
        cout<<T -> Key<<" ";
        LNR(T -> pRight);
    }
}
```

2b:

2*x+y*5*a-b*5*a-b*5*a-b*5*a-b

Nội dung 2:

Câu 1:

1a:

<u>Bước 1:</u>

i = 1;

<u>Bước 2:</u>

j = N; **Trong khi (j > i) thực hiện:** **Nếu a[j]<a[j-1]: Hoán vị a[j] và a[j-1]** **j = j-1;**
<u>Bước 3:</u> **i = i+1;** **Nếu i >N-1: Hết dãy. Dừng** **Ngược lại: Lặp lại Bước 2.**
1b:
Bước 1: 81 38 48 22 69 93 14 45 **13**
Bước 2: 81 48 38 69 93 22 45 **14 13**
Bước 3: 81 48 69 93 38 45 **22 14 13**
Bước 4: 81 69 93 48 45 **38 22 14 13**
Bước 5: 81 93 69 48 **45 38 22 14 13**
Bước 6: 93 81 69 **48 45 38 22 14 13**
Bước 7:

93 81 **69 48 45 38 22 14 13**
Bước 8: 93 **81 69 48 45 38 22 14 13**
Dãy đã sắp xếp giảm dần: **93 81 69 48 45 38 22 14 13**

Câu 2:

2a:

``
PNODE CreateNode()
{
PNODE p;
SINHVIEN t;
printf("Nhap vao MSSV");
scanf("%d",&t.MSSV);
fflush(stdin);
printf("Nhap vao Ten");
gets(t.HoTen);
printf("Nhap vao DTB");
scanf("%f",&t.DTB);
p = (PNODE)malloc(sizeof(NODE));//p->SV = t;
p->SV.MSSV = t.MSSV;
p->SV.DTB = t.DTB;
strcpy(p->SV.HoTen,t.HoTen);
p->pNext = NULL;
return p;
}
``
``
void AddTailHead(PNODE q)
{
if (pHead==NULL)

```
    {
        pHead = q;
    }
    else
    {
        p = pHead;
        while (p->pNext==NULL)
            p=p->pNext;
        p->pNext =q;
    )
}
```

```
void Nhap()
{
    for (int i=0;i<10;i++)
    {
        AddHead(CreateNode());
    }
}
```

Câu b:

```
void In()
{
    PNODE p;
    p=pHead;
    while (p!=NULL)
    {
        if (p->SV.DTB<5)
        {
            printf("%d\t%s\t%f",p->SV.MSSV,
                        p->SV.HoTen,p->SV.DTB);
        }
        p=p->pNext;
```

```
        }
}
```

2c:

```
void GPDSD()
{
        PNODE p=pHead;
        while (pHead!=NULL)
        {
                p=pHead;
                pHead=pHead->pNext;
                free(p);
        }
}
```

2d:

```
main()
{
        Nhap();
        In();
        GPDSD();
        getch();
}
```

Nội dung 3

Câu 1:

1a: Trong phần này, sinh viên có thể có nhiều cách viết, có nhiều cách return (nhưng phải theo cặp giá trị flase/true, 0/1 hoặc -1/i). Khi chấm **Giảng viên cần lưu ý là mảng đang xét là mảng giảm dần**.

```
int BinarySearch(int a[],int n,int x)
{
        int left, right, mid; left=0; right=n-1;
```

```
        do{
            mid=(left+right)/2;
            if(a[mid]==x)
                return 1;
            else  if(a[mid]> x)
                left=mid+1;
            else
                right=mid-1;
        }while(left<=right);
```

```
        return 0;
}
```

1b: Trường hợp này tìm không thấy, vòng lặp dừng khi left>right

Chỉ số mảng:	0	1	2	3	4	5
Mãng a:	**{12**	**10**	**8**	**6**	**4**	**2}**

Lần 1: left=0, right=5, mid=(left+right)/2=2;
a[mid]=8 > x

→ **left=mid+1=3**

Lần 2: left=3, right=5, mid=(left+right)/2=4;
a[mid]=4 < x

→ **right=mid - 1=3**

Lần 3: left=3, right=3, mid=(left+right)/2=3;
a[mid]=6 > x

→ **left=mid + 1=4**

Lần 4: left=4, right=3;

giá trị left > right

nên vòng lặp kết thúc và kết luận không tìm thấy x trong a

Câu 2:

2a:

```
StackNode* CreateStackNode(ItemType x)
{
     StackNode* p = new StackNode;
     if(p == NULL)
     {
       printf("Không đủ bộ nhớ để cấp phát!");
       getch();
       return NULL;
     }
     p->Info = x;
     p->Next = NULL;
     return p;
}
```

2b:

```
void InitStack(Stack &s)
{
     s.Top = NULL;
}
```

2c:

```
int IsEmpty(Stack s)
{
     return (s.Top == NULL);
}
```

2d:

```
int Push(Stack &s, StackNode* p)
```

```
{
	if(p == NULL)
		return 0; //Không thực hiện được
	if(IsEmpty(s) == 1)
		s.Top = p;
	else
	{
		p->Next = s.Top;
		s.Top = p;
	}
	return 1; //Thực hiện thành công
}
```

2e:

```
int Pop(Stack &s, ItemType &x)
{
	if(IsEmpty(s) == 1) //Stack rỗng
		return 0; //Không thực hiện được
	StackNode* p = s.Top;
	s.Top = s.Top->Next;
	x = p->Info; //Lấy thông tin của nút ở đỉnh Stack
	delete p; //Hủy nút do p trỏ đến
	return 1; //Thực hiện thành công
}
```

Nội dung 4:

Câu 1:

1a:

Bước 1: i = 1;
Bước 2: Tìm phần tử a[vtmin] nhỏ nhất trong dãy hiện hành từ a[i] đến a[N]
Bước 3: Hoán vị a[vtmin] và a[i]
Bước 4: i = i+1 Nếu I < N thì lặp lại Bước 2 Ngược lại: Dừng.
1b:
Chỉ số mảng: 0 1 2 3 4 **Mảng a: 64 25 12 22 11**

// Tìm phần tử nhỏ nhất trong a[0 ... 4]
// và đặt nó ở đầu
11 25 12 22 64

// Tìm phần tử nhỏ nhất trong a[1 ... 4]
// và đặt nó ở đầu arr [1 ... 4]
11 12 25 22 64

// Tìm phần tử nhỏ nhất trong a[2 ... 4]
// và đặt nó ở đầu a[2 ... 4]
11 12 22 25 64

// Tìm phần tử nhỏ nhất trong a[3 ... 4]
// và đặt nó ở đầu a[3 ... 4]
11 12 22 25 64

Câu 2:

2a:

```
QNode* CreateQNode(int x)
{
	QNode* p = new QNode;
	if(p == NULL)
	{
		printf("khong du bo nho");
		return NULL;
	}
	p->Info = x;
	p->Next = NULL;
	return p;
}
```

2b:

```
void InitQueue(Queue &q)
{
	q.Head = NULL;
	q.Tail = NULL;
}
```

2c:

```
int IsEmpty(Queue q)
{
	if(q.Head == NULL)
		return 1;
	return 0;
}
```

2d:

```
void InsertTail(Queue &q, int x)
{
	QNode *p=CreateQNode(x);
	if(p == NULL)
	return ;
	if(IsEmpty(q) == 1)
		q.Head = p;
	else
		q.Tail->Next = p;
		q.Tail = p;
}
```

2e:

```
int Remove(Queue &q)
{
	int x;
	if(IsEmpty(q) == 1)
		return 0; //Thực hiện không thành công
	QNode* p = q.Head;
	x = p->Info; //Lấy thông tin của nút bị hủy
	q.Head = q.Head->Next;
	if(q.Head == NULL)
		q.Tail = NULL;
	return x;
}
```

Nội dung 5

Câu 1:

1a:

```
void partition(double b[], int m, int& pPos)
{
	double pivot = b[0];
	int left = 1, right = m - 1;
	while(left <= right)
	{
		while(left <= right && b[left] >= pivot)
			left++;
		while(left <= right && b[right] < pivot)
			right--;
		if(left < right)
		{
			swap(b[left], b[right]);
			left++;
			right--;
		}
	}
	swap(b[0], b[right]);
	pPos = right;
}
```

```
void quicksort(double a[], int n)
{
	if(n <= 1)
		return;
	int pivotPos;
	partition(a, n, pivotPos);
	quicksort(a, pivotPos);
```

```
	quicksort(a + pivotPos + 1, n - pivotPos - 1);
}
```

1b:

Trong hàm phân hoạch, khi chốt chia mảng n phần tử thành một phần không có phần tử nào, một phần chứa n-1 phần tử thì có thời gian chạy xấu nhất.

Ví dụ: Đầu vào là dãy có thứ tự ngược với thứ tự yêu cầu

(1, 3, 5, 7, 8).

1c:

Ta có: $T(n) = O(n) + T(0) + T(n-1) = O(n^2)$

Câu 2:

2a:

```
QNode* CreateQNode(int x)
{
	QNode* p = new QNode;
	if(p == NULL)
	{
		printf("khong du bo nho");
		return NULL;
	}
	p->Info = x;
	p->Next = NULL;
	return p;
}
```

2b:

```
void InitQueue(Queue &q)
{
	q.Head = NULL;
	q.Tail = NULL;
}
```

2c:

```
int IsEmpty(Queue q)
{
	if(q.Head == NULL)
		return 1;
	return 0;
```

```
}
```

Nội dung 6:

Câu 1:

1a: Trong phần này, sinh viên có thể có nhiều cách viết, có nhiều cách return (nhưng phải theo cặp giá trị flase/true, 0/1 hoặc -1/i). Khi chấm **Giảng viên cần lưu ý là mảng đang xét là mảng tăng dần**.

```
int BinarySearch(int a[],int n,int x)
{
        int left, right, mid; left=0; right=n-1;
        do{
                mid=(left+right)/2;
                if(a[mid]==x)
                        return 1;
                else  if(a[mid] < x)
                        left=mid+1;
                    else
                        right=mid-1;
        }while(left<=right);
        return 0;
}
```

1b: Trường hợp này tìm không thấy, vòng lặp dừng khi left>right

Chỉ số mảng:	0	1	2	3	4
Mãng a:	**{**42	62	65	81	90}

Lần 1: left=0, right=4, mid=(left+right)/2=2;
a[mid]=65 < X

→ **left=mid+1=3**

Lần 2: left=3, right=4, mid=(left+right)/2=3;
a[mid]=81 < X

→ **right=mid + 1=4**

Lần 3: left=4, right=4, mid=(left+right)/2=4;

a[mid]=90 < X

→ **left=mid + 1=4**

left=4, right=4;

giá trị left = right

nên vòng lặp kết thúc và kết luận không tìm thấy X trong a

1c:

Trường hợp số lần so sánh ít nhất

Ví dụ: Tìm phần tử 65 chỉ 1 lần so sách a[mid]=65

Chỉ số mảng:		0	1	2	3	4
Mãng a:	**{**42	62	65	81	90}	

Trường hợp số lần so sánh nhiều nhất.

Ví dụ: Tìm phần tử 90 có 3 lần so sách a[mid]=90.

Chỉ số mảng:		0	1	2	3	4
Mãng a:	**{**42	62	65	81	90}	

Câu 2:

2a:

```
PNODE CreateNode()
{
    PNODE p;
    SINHVIEN t;
    printf("Nhap vao MSSV");
    scanf("%d",&t.MSSV);
    fflush(stdin);
    printf("Nhap vao Ten");
```

```
    gets(t.HoTen);
    printf("Nhap vao DTB");
    scanf("%f",&t.DTB);
    p = (PNODE)malloc(sizeof(NODE));//p->SV = t;
    p->SV.MSSV = t.MSSV;
    p->SV.DTB = t.DTB;
    strcpy(p->SV.HoTen,t.HoTen);
    p->pNext = NULL;
    return p;
}
```

```
void AddTailHead(PNODE q)
{
    if (pHead==NULL)
    {
        pHead = q;
    }
    else
    {
        p = pHead;
        while (p->pNext==NULL)
            p=p->pNext;
        p->pNext =q;
    )
}
```

```
void Nhap()
{
    for (int i=0;i<10;i++)
    {
        AddHead(CreateNode());
```

```
	}
}
```

2b:

```
void In()
{
	PNODE p;
	p=pHead;
	while (p!=NULL)
	{
		if (p->SV.DTB>=5)
		{
			printf("%d\t%s\t%f",p->SV.MSSV,
						p->SV.HoTen,p->SV.DTB);
		}
		p=p->pNext;
	}
}
```

2c:

```
void GPDSD()
{
	PNODE p=pHead;
	while (pHead!=NULL)
	{
		p=pHead;
		pHead=pHead->pNext;
		free(p);
	}
}
```

Nội dung 7

Câu 1:

1a:
Bước 1: i = 1;
Bước 2: Tìm phần tử a[vtmin] nhỏ nhất trong dãy hiện hành từ a[i] đến a[N]
Bước 3: Hoán vị a[vtmin] và a[i]
Bước 4: i = i+1 Nếu I < N thì lặp lại Bước 2 Ngược lại: Dừng.
1b:
Viết đúng các bước đưa từng phần tử nhỏ nhất lên trước (2,5 điểm). Nếu SV làm sai phương pháp thì không tính điểm. Nếu SV lỡ nhầm sai 1-2 trường hợp thì chỉ cho 1.0 điểm

Câu 2:

2a:

```
StackNode* CreateStackNode(ItemType x)
{
     StackNode* p = new StackNode;
     if(p == NULL)
     {
       printf("Không đủ bộ nhớ để cấp phát!");
       getch();
       return NULL;
     }
     p->Info = x;
     p->Next = NULL;
     return p;
}
```

2b:

```
void InitStack(Stack &s)
{
     s.Top = NULL;
}
```

2c:

```
int IsEmpty(Stack s)
```

```
{
	return (s.Top == NULL);
}
```

2d

```
int Push(Stack &s, StackNode* p)
{
	if(p == NULL)
		return 0; //Không thực hiện được
	if(IsEmpty(s) == 1)
		s.Top = p;
	else
	{
		p->Next = s.Top;
		s.Top = p;
	}
	return 1; //Thực hiện thành công
}
```

Nội dung 8:

Câu 1:

1a:

Nếu sai 1 lỗi trừ đi 0.25 điểm.

2*x+y*5*a–b*5*a–b*5*a–b*5*a-b

1b:

Nếu sai 1 lỗi trừ đi 0.25 điểm.

*+*2xy**-*5ab-*5ab-*5ab

Câu 2:

2a:

Nếu sai 1 lỗi trừ 0.25 đ

```
void insertionSort(int arr[], int n)
{
  int i, key, j;
```

```
  for (i = 1; i < n; i++)
  {
     key = arr[i];
     j = i-1;

     /* Di chuyển các phần tử có giá trị lớn hơn giá trị
     key về sau một vị trí so với vị trí ban đầu
     của nó */
     while (j >= 0 && arr[j] > key)
     {
        arr[j+1] = arr[j];
        j = j-1;
     }
     arr[j+1] = key;
  }
}
```

2b:

{4, **3**, 2, 10, 12, 1, 5, 6}

{**3**, 4, **2**, 10, 12, 1, 5, 6}

{2, 3, 4, **10**, 12, 1, 5, 6}

{2, 3, 4, 10, **12**, 1, 5, 6}

{2, 3, 4, 10, 12, **1**, 5, 6}

{1, 2, 3, 4, 10, 12, **5**, 6}

{1, 2, 3, 4, 5, 10, 12, **6**}

{1, 2, 3, 4, 5, 6, 10, 12}

Mảng a đã sắp xếp tăng:

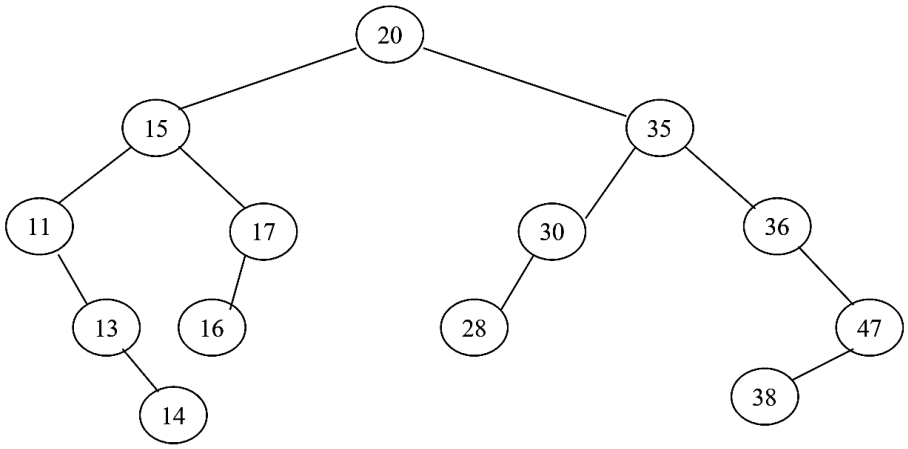
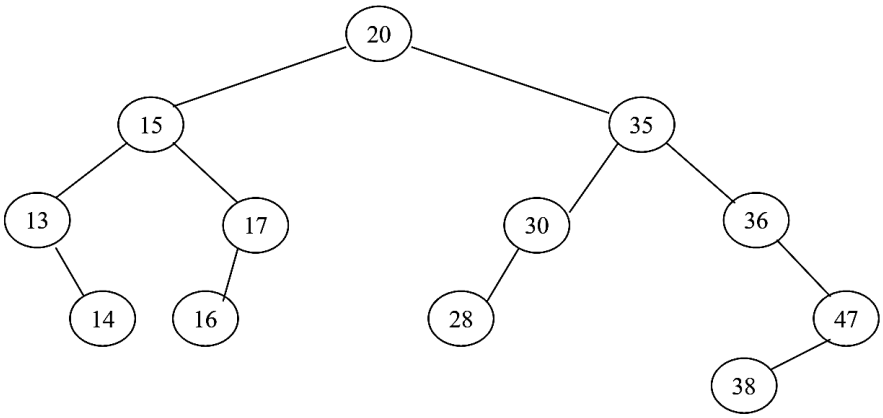
Câu 3:

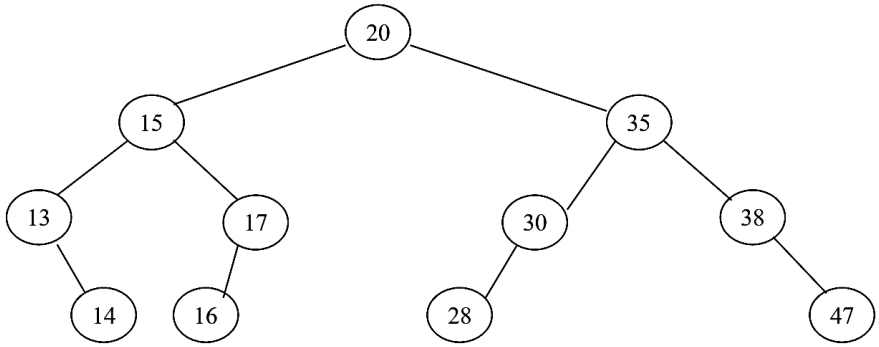
Kết quả: Nếu sai 1 lỗi trừ 0.25 đ

S = ket-doan

Nội cung 9:

Câu 1:

1a:
Vẽ cây nhị phân tìm kiếm cho dãy số trên: Sai mỗi lỗi trừ 0.25 đ 
1b:
Trình bày từng bước và vẽ lại cây sau khi lần lượt xoá các nút: **11** và **36.**   **<u>Xóa nút 11:</u>** 
<u>Xóa nút 36:</u>



Câu 2:

2a:

```
void swap(int &x, int &y) // Hàm hoán đổi
{
   int temp = x;
   x = y;
   y = temp;
}
void Bubble_Sort(int a[],int n) // Hàm sắp xếp
{
        int i, j;
        for (i=0; i<n-1; i++){
                for (j=n-1; j>i; j--){
                        if(a[j]<a[j-1])
                        {       //nếu sai vị trí thì đổi chỗ
                                swap(a[j], a[j-1]);
                        }
                }
        }
}
```

2b:

Mảng a: Sai mỗi bước trừ đi 0.25 đ

38, 81, 22, 48, 13, 69, 93, 14, 45

Bước 1: 81 38 48 22 69 93 14 45 **13**
Bước 2: 81 48 38 69 93 22 45 **14 13**
Bước 3: 81 48 69 93 38 45 **22 14 13**
Bước 4: 81 69 93 48 45 **38 22 14 13**
Bước 5: 81 93 69 48 **45 38 22 14 13**
Bước 6: 93 81 69 **48 45 38 22 14 13**
Bước 7: 93 81 **69 48 45 38 22 14 13**
Bước 8: 93 **81 69 48 45 38 22 14 13**

Câu 3:

Kết quả: Nếu sai 1 lỗi trừ 0.25 đ

x = leolh

Nội dung 10:

Câu 1:

1a:

```
QNode* CreateQNode(int x)
{
	QNode* p = new QNode;
	if(p == NULL)
	{
		printf("khong du bo nho");
		return NULL;
	}
	p->Info = x;
	p->Next = NULL;
	return p;
}
```

1b:

```
void InitQueue(Queue &q)
{
	q.Head = NULL;
	q.Tail = NULL;
}
```

1c:

```
int IsEmpty(Queue q)
{
	if(q.Head == NULL)
		return 1;
	return 0;
}
```

1d:

```
void InsertTail(Queue &q, int x)
{
	QNode *p=CreateQNode(x);
```

```
	if(p == NULL)
	return ;
	if(IsEmpty(q) == 1)
		q.Head = p;
	else
		q.Tail->Next = p;
		q.Tail = p;
}
```

1e:

```
int Remove(Queue &q)
{
	int x;
	if(IsEmpty(q) == 1)
		return 0; //Thực hiện không thành công
	QNode* p = q.Head;
	x = p->Info; //Lấy thông tin của nút bị hủy
	q.Head = q.Head->Next;
	if(q.Head == NULL)
		q.Tail = NULL;
	return x;
}
```

Câu 2:

```
void swap(int &x, int &y) // Hàm hoán đổi
{
   int temp = x;
   x = y;
   y = temp;
}
void Bubble_Sort(int a[],int n) // Hàm sắp xếp
{
        int i, j;
        for (i=0; i<n-1; i++){
                for (j=n-1; j>i; j--){
                        if(a[j]<a[j-1])
```

```
                    {       //nếu sai vị trí thì đổi chỗ
                            swap(a[j], a[j-1]);
                    }
            }
    }
}
```

Câu 3:

Bước 1: 81 38 48 22 69 93 14 45 **13**
Bước 2: 81 48 38 69 93 22 45 **14 13**
Bước 3: 81 48 69 93 38 45 **22 14 13**
Bước 4: 81 69 93 48 45 **38 22 14 13**
Bước 5: 81 93 69 48 **45 38 22 14 13**
Bước 6: 93 81 69 **48 45 38 22 14 13**
Bước 7: 93 81 **69 48 45 38 22 14 13**
Bước 8: 93 **81 69 48 45 38 22 14 13**